Generated benchmark artifacts

Typeset from `paper/generated/x86_64/` by `benchmarks/scripts/build_pdf.py`. Every number resolves to a row in the result set named below; see `PROVENANCE.md` for the inputs, transformation and caveats of each artifact.

`result set:` current
`commit:` 18d0f83627b5
`host:` a2
`measured:` 2026-09-12
`inputs:` sha256:51d37586d0e48420

Dataset	Tool	Metric	-@	Mapped	Mapq 60	Missed (%)	Wrong Q60	Index (s)	Map (s)	Mem (GB)
B01	shmap-rs	Containment	1	149 194	139 309	6.78	—	7.07	32.32	2.65
B01	shmap-rs	Jaccard	1	146 120	138 421	7.37	—	7.03	45.91	2.62
B01	shmap-rs	bucket_SH	1	149 236	138 250	7.49	—	7.09	25.07	2.65
B01	cpp-shmap	Containment	1	149 194	139 309	6.78	—	34.91	74.41	18.85
B01	cpp-shmap	Jaccard	1	146 120	138 421	7.37	—	35.61	101.16	18.85
B01	cpp-shmap	bucket_SH	1	149 236	138 250	7.49	—	35.73	57.82	18.85
B01	mapquik	—	32	148 225	—	—	—	—	18.00	18.14
B01	winnowmap2	—	32	189 022	—	—	—	—	1328.30	9.78
B02	shmap-rs	Containment	1	125 000	117 532	5.97	0	7.04	25.07	2.57
B02	shmap-rs	Jaccard	1	125 000	119 065	4.75	6	6.97	40.45	2.62
B02	shmap-rs	bucket_SH	1	125 000	116 800	6.56	1	7.06	21.78	2.60
B02	cpp-shmap	Containment	1	125 000	117 532	5.97	—	31.16	52.73	18.85
B02	cpp-shmap	Jaccard	1	125 000	119 065	4.75	—	31.11	79.69	18.85
B02	cpp-shmap	bucket_SH	1	125 000	116 802	6.56	—	31.05	46.44	18.85
B02	mapquik	—	32	124 221	—	—	—	—	19.40	18.09
B02	winnowmap2	—	32	129 488	—	—	—	—	1006.40	9.40
B03	shmap-rs	Containment	1	241 988	227 521	6.19	—	7.09	41.39	2.58
B03	shmap-rs	Jaccard	1	241 262	228 887	5.63	—	7.03	55.14	2.65
B03	shmap-rs	bucket_SH	1	242 047	226 931	6.43	—	7.11	31.56	2.62
B03	cpp-shmap	Containment	1	241 991	227 521	6.19	—	36.93	84.54	18.85
B03	cpp-shmap	Jaccard	1	241 265	228 886	5.63	—	36.10	113.22	18.85
B03	cpp-shmap	bucket_SH	1	242 050	226 934	6.43	—	36.25	65.61	18.85
B03	mapquik	—	32	239 539	—	—	—	—	18.10	18.17
B03	winnowmap2	—	32	280 093	—	—	—	—	997.00	8.04
B04	shmap-rs	Containment	1	2 419 767	2 273 103	6.28	—	7.06	399.36	2.60
B04	shmap-rs	Jaccard	1	2 411 433	2 286 067	5.74	—	6.95	551.59	2.67
B04	shmap-rs	bucket_SH	1	2 420 302	2 267 208	6.52	—	7.04	315.63	2.65
B04	cpp-shmap	Containment	1	2 419 796	2 273 121	6.28	—	37.79	852.89	18.85
B04	cpp-shmap	Jaccard	1	2 411 461	2 286 083	5.74	—	34.27	1125.60	18.85
B04	cpp-shmap	bucket_SH	1	2 420 331	2 267 238	6.52	—	35.77	657.25	18.85
B04	mapquik	—	32	2 394 360	—	—	—	—	43.90	18.11
B04	winnowmap2	—	32	2 808 979	—	—	—	—	9019.50	8.38
B05	shmap-rs	Containment	1	39 608	37 839	58.97	—	7.10	14.12	2.60
B05	shmap-rs	Jaccard	1	5 956	5 409	94.13	—	7.02	15.13	2.57
B05	shmap-rs	bucket_SH	1	41 115	39 226	57.46	—	7.08	11.08	2.62
B05	cpp-shmap	Containment	1	39 608	37 839	58.97	—	31.14	26.11	18.85
B05	cpp-shmap	Jaccard	1	5 956	5 409	94.13	—	31.09	30.02	18.85
B05	cpp-shmap	bucket_SH	1	41 115	39 225	57.47	—	31.12	24.06	18.85
B05	winnowmap2	—	32	99 107	—	—	—	—	331.50	6.15
B06	bwa-mem2	—	32	41 807 925	—	—	—	—	534.50	35.74

Dataset	Metric	Matches per read			Potential		Buckets per read	
		possible	examined	in mapping	realized	unrealized	seeded	final
B01	Containment	113 462	2 376	206	47.8×	11.6×	194.0	2.38
B01	Jaccard	113 462	2 376	203	47.8×	11.7×	194.0	2.31
B01	bucket_SH	113 462	2 376	208	47.8×	11.4×	194.0	2.28
B02	Containment	113 804	2 907	209	39.2×	13.9×	206.2	2.29
B02	Jaccard	113 804	2 907	208	39.2×	13.9×	206.2	2.29
B02	bucket_SH	113 804	2 907	211	39.2×	13.8×	206.2	2.26
B03	Containment	70 179	1 886	117	37.2×	16.1×	251.8	2.35
B03	Jaccard	70 179	1 886	117	37.2×	16.1×	251.8	2.34
B03	bucket_SH	70 179	1 886	119	37.2×	15.9×	251.8	2.24
B04	Containment	69 463	1 938	116	35.8×	16.6×	250.3	2.35
B04	Jaccard	69 463	1 938	116	35.8×	16.7×	250.3	2.34
B04	bucket_SH	69 463	1 938	118	35.8×	16.4×	250.3	2.25
B05	Containment	67 358	307	51	219.6×	6.0×	84.8	0.99
B05	Jaccard	67 358	307	9	219.6×	33.1×	84.8	0.17
B05	bucket_SH	67 358	307	54	219.6×	5.7×	84.8	1.00

Table 2: Efficiency of the seed heuristic. Realized potential is possible matches divided by matches examined (work avoided); unrealized potential is matches examined divided by matches inside the reported mapping (work still wasted).

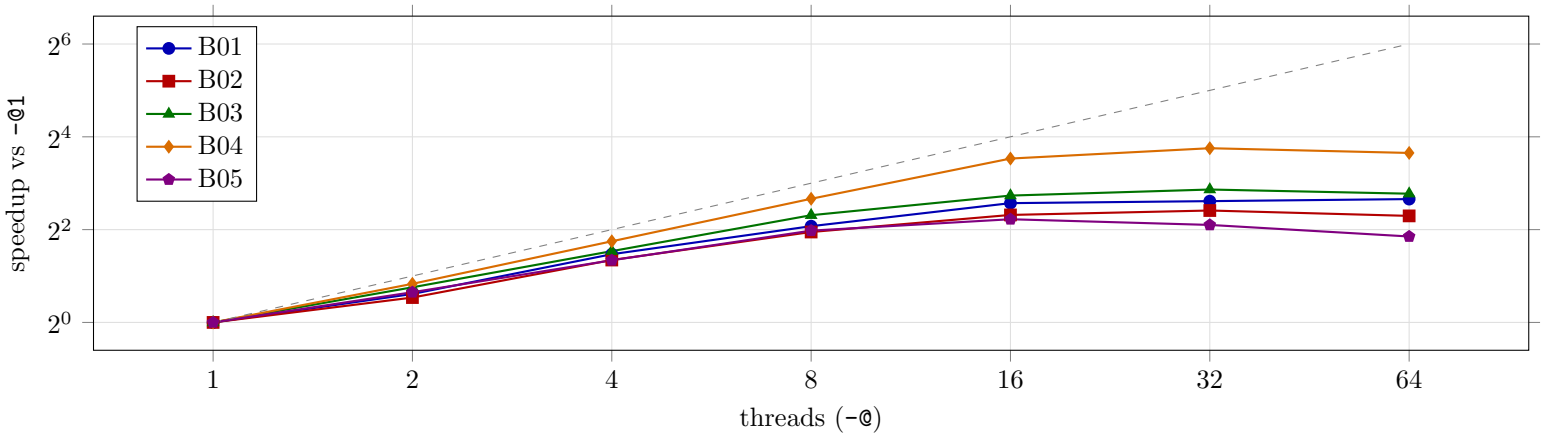

Figure 1: Whole-run speedup against thread count, Containment. The dashed line is linear speedup.

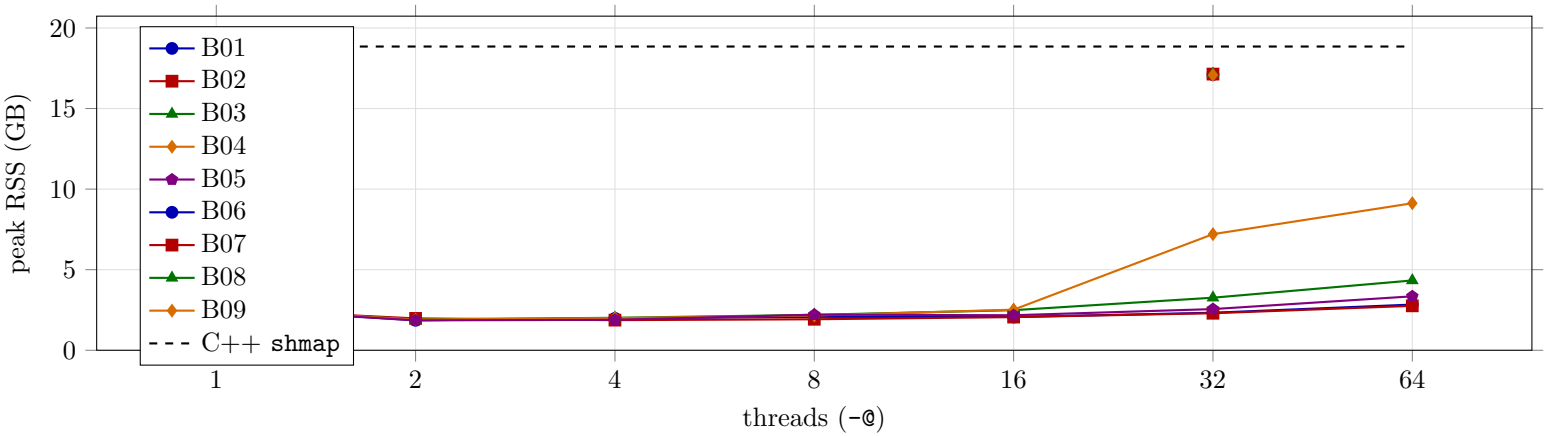

Figure 2: Peak resident memory against thread count, Containment, with the C++ reference as a horizontal line.

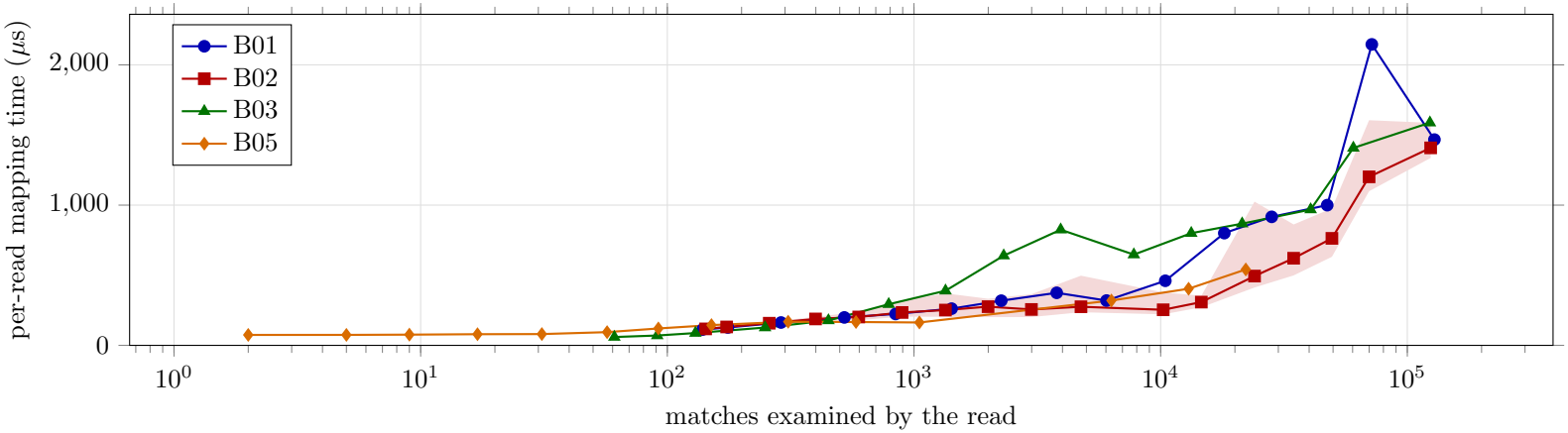

Figure 3: Per-read mapping time against the number of matches the read examined, Containment. Points are per-bin medians over reads grouped into 18 logarithmic bins; the shaded band is the interquartile range for the simulated set.

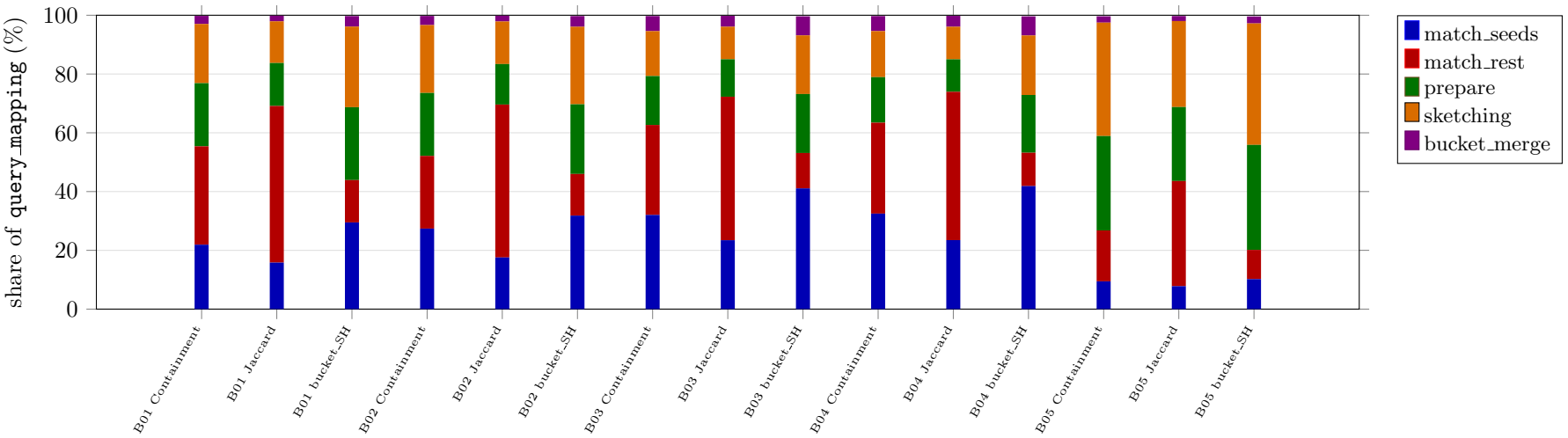

Figure 4: Where mapping time goes: stage shares of `query_mapping`, single-threaded.

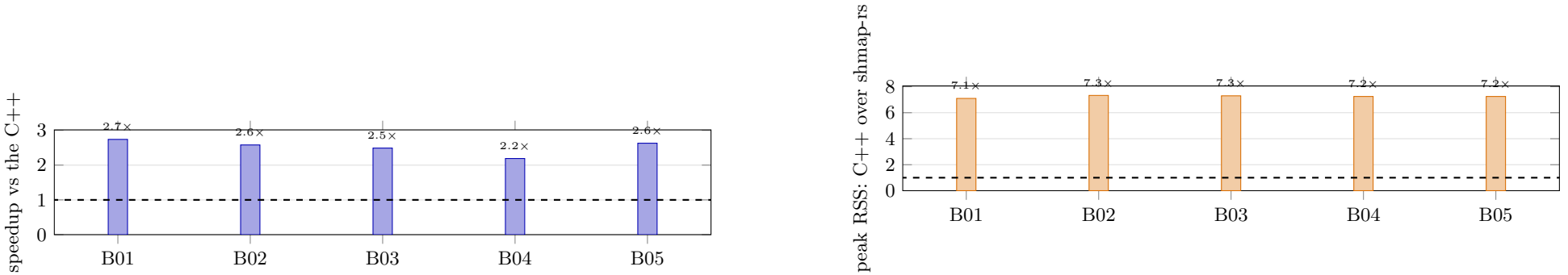

Figure 5: The two headline results, per benchmark, single-threaded on both sides: wall-clock speedup over the C++ (top) and how many times less peak memory the port uses (bottom). The dashed line is parity.

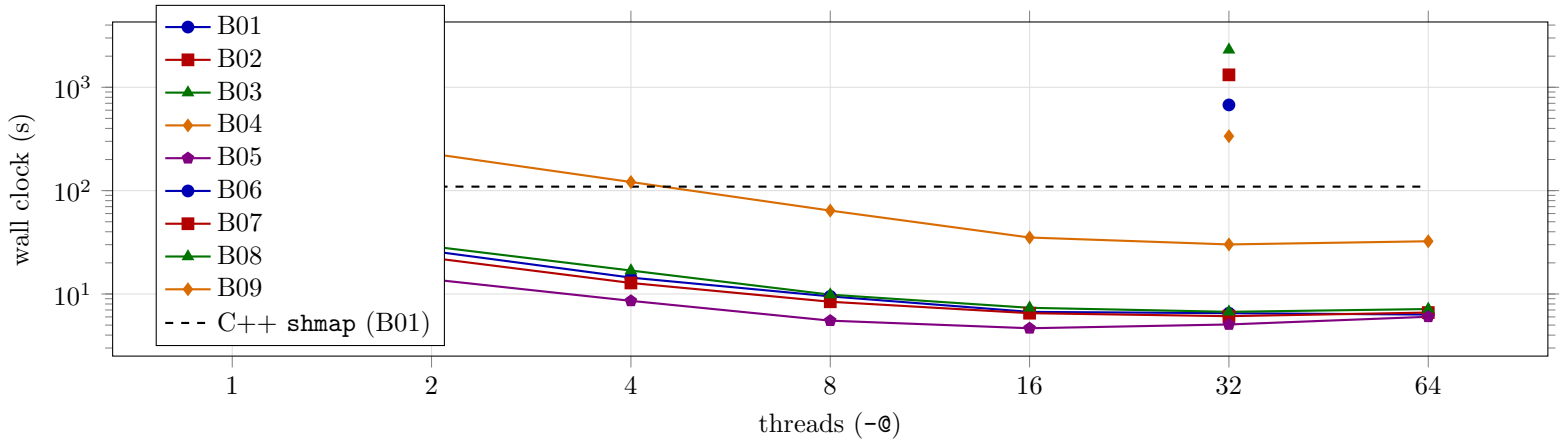

Figure 6: Wall clock against thread count, Containment, with the C++ reference as a horizontal line. The gap at `-@1` is the single-threaded speedup the summary quotes; everything to the right of it is threading the C++ does not have.